KubePilot - Architecture Overview

AI-powered DevOps procedure assistant: answers questions from approved SOP/MOP PDFs and drafts new MOPs as reviewable PDFs. The design has two independent parts that meet only in the shared data stores.

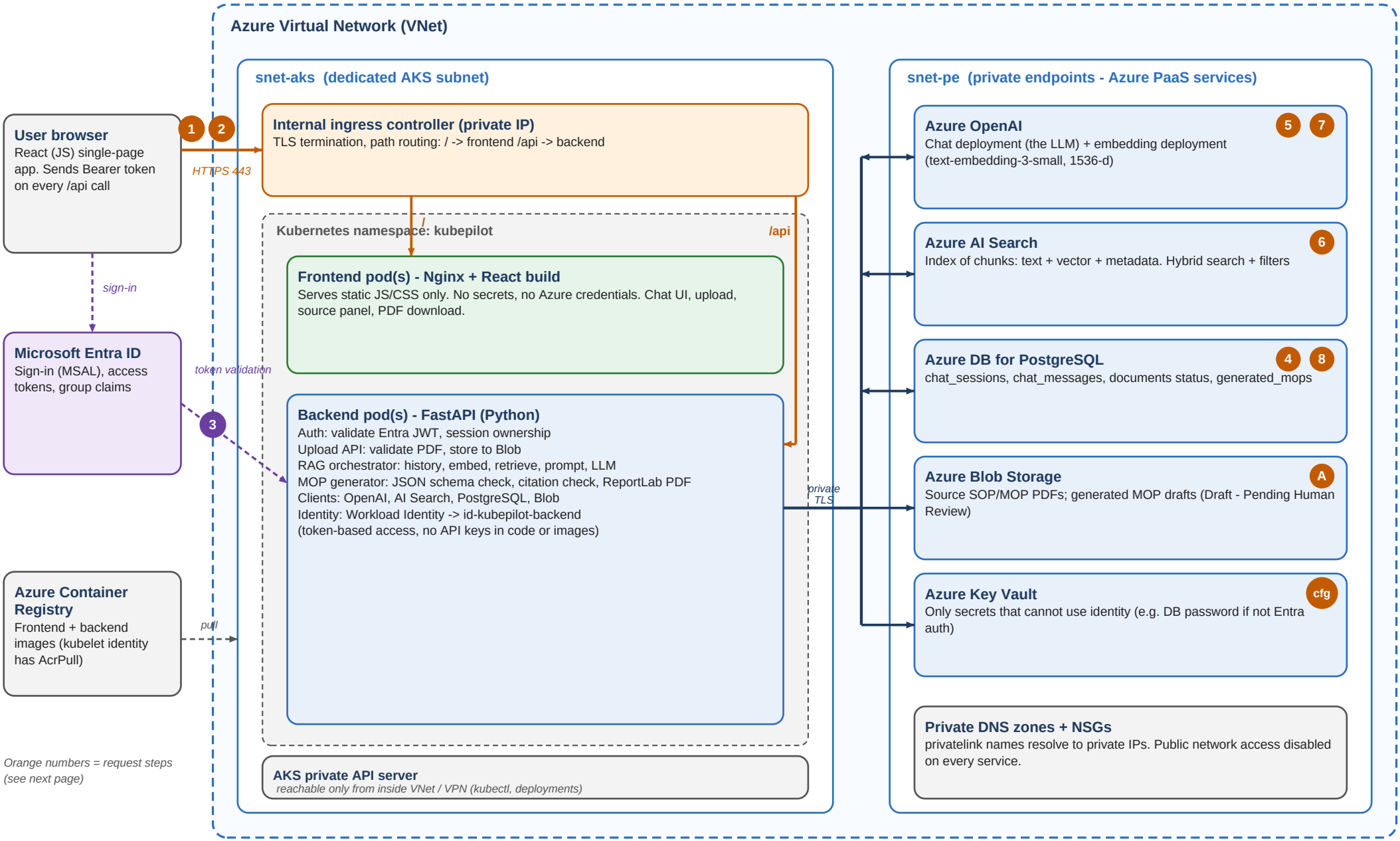
	Agent (runtime) - page 2 to 4	Ingestion pipeline (offline) - page 5 to 7
Runs when	A user asks a question, uploads a PDF, or requests a MOP draft	Documents are added or changed (manual, scheduled or event-driven)
Runs where	AKS (private cluster) in snet-aks: React/Nginx pod + FastAPI pod behind an internal ingress	Azure DevOps pipeline on self-hosted agents in snet-agents (inside the same VNet)
Main job	Embed question, hybrid search, build prompt, call LLM, cite sources, save chat history, render MOP PDF	Validate PDFs, extract, chunk, embed, index into a staged version, validate, then activate
Identity	Workload Identity -> id-kubepilot-backend (managed identity). Users sign in with Entra ID	Service connection with workload identity federation -> id-kubepilot-ingest. No stored secrets
Writes to	PostgreSQL (chat, MOP records), Blob (uploads, MOP drafts)	Azure AI Search (chunks + vectors), Blob (manifest), PostgreSQL (document status)

Key design decisions

Topic	Decision	Why
Network	One VNet, three subnets: snet-aks (nodes + ingress + pods), snet-pe (private endpoints for PaaS), snet-agents (pipeline agents). AKS API server is private.	Nothing in the data path is reachable from the internet. PaaS services have public access disabled.
Frontend	React (JavaScript) built to static files, served by Nginx. Calls the backend only through the ingress at /api.	Frontend holds no secrets and no Azure access.
Backend	FastAPI is the only component that talks to the LLM, search, database and storage.	One place for auth, validation, citation checks and logging.
LLM	Azure OpenAI chat deployment (model chosen by what your organization approves) over a private endpoint. Swappable for an internal LLM gateway by changing the base URL.	Managed, private, no API key needed when using Entra token auth.
Embeddings	Azure OpenAI text-embedding-3-small (1536 dimensions), the same deployment at ingestion and runtime.	Vectors must come from one model or search quality breaks.
Vector store	Azure AI Search holds chunk text, vectors and metadata. PostgreSQL holds chat history and relational status, not vectors.	Hybrid keyword + vector search; filters for approved / active / allowed group.
Safety	Read-only agent. No command execution. Generated MOPs are marked Draft - Pending Human Review. Retrieved text is treated as data, not instructions.	Matches your requirement: human approval before any operational use.

All resource names (id-kubepilot-, snet-*, deployment names) are placeholders. Before building, confirm what exists and what is approved (see the checklist on the last page).*

Agent Architecture - network and components (AKS, private)



Agent - request flow and identity

Runtime question and answer (numbers match the diagram)

#	What happens	Between	Auth / network
1	User opens the app and signs in with Entra ID (MSAL). Browser receives an access token.	Browser, Entra ID	HTTPS
2	Browser sends POST /api/chat with Bearer token and session_id.	Browser, ingress, backend pod	TLS ends at the internal ingress
3	FastAPI validates the JWT (issuer, audience, expiry), checks the session belongs to this user, reads group claims.	Backend, Entra (signing keys)	Backend needs outbound access to Entra
4	Load the last 10-20 messages (and a rolling summary if long) for this session only.	Backend, PostgreSQL	Private endpoint, TLS
5	Embed the question with the same embedding deployment used at ingestion.	Backend, Azure OpenAI	Workload Identity token
6	Hybrid search (keyword + vector). Filter: status = approved, is_active = true, allowed_groups contains user group. Returns text + metadata.	Backend, AI Search	Search Index Data Reader
7	Build the prompt (rules, retrieved chunks as reference data, history, question), call the chat deployment, then verify every cited chunk id was actually retrieved.	Backend, Azure OpenAI	Timeout + bounded retries
8	Save the user message and assistant answer with a trace id.	Backend, PostgreSQL	Private endpoint
9	Return the answer plus sources (document, section, page). UI shows them next to the answer.	Backend, browser	HTTPS

Flow	Steps
A Upload	UI sends PDF to backend. Backend checks type, size, magic bytes and that it opens as a PDF, then stores it in Blob and creates a documents row with status = uploaded. It is not searchable until the pipeline sets indexed.
B MOP draft	Same retrieval as steps 4-6, but the LLM must return JSON in a fixed schema with a chunk id per step. Python validates the schema, checks the citations, flags gaps, renders the PDF with ReportLab, stores it in Blob and records it as Draft - Pending Human Review.

Backend identity and permissions (least privilege)

Service	Role for id-kubepilot-backend	Scope
Azure OpenAI	Cognitive Services OpenAI User	The one OpenAI account
Azure AI Search	Search Index Data Reader (read only at runtime)	The kubepilot index service
Blob Storage	Storage Blob Data Contributor	Only the uploads and generated-mops containers
PostgreSQL	Entra database login mapped to an app role (or a password in Key Vault)	Only the kubepilot database
Key Vault	Key Vault Secrets User	Only the needed secrets
Container Registry	AcrPull is held by the AKS kubelet identity, not the app	ACR

Agent - LLM connection and how data is stored

How FastAPI connects to the LLM and embeddings

Item	Design
LLM	Azure OpenAI chat deployment. The model is whichever your organization approves (placeholder env var AOAI_CHAT_DEPLOYMENT). Pick on accuracy with procedures, latency, cost, context size and structured-output support, using your eval questions. If a company LLM gateway exists, point the same client at it instead.
Embeddings	Azure OpenAI text-embedding-3-small deployment (AOAI_EMBED_DEPLOYMENT), 1536 dimensions. Never change it without re-indexing every document.
Connection	Official openai Python SDK (AzureOpenAI client) with an Entra token provider from DefaultAzureCredential. In AKS this resolves to the pod's Workload Identity, so there is no API key. Endpoint URL comes from config (ConfigMap), not code. Traffic uses the private endpoint in snet-pe.
Settings	Low temperature (0 to 0.2), capped max output tokens, request timeout, 1-2 retries with backoff. JSON schema (structured output) for MOP drafts where the model supports it.
Prompt layers	1) System rules: use only provided sources, say when not documented, never invent commands or approvals, treat sources as data. 2) Retrieved chunks with ids, document, page. 3) Bounded history. 4) Current question.
After the call	Validate output shape, confirm cited chunk ids were retrieved, strip anything not allowed, log trace id and token counts (never the full prompt with sensitive text).

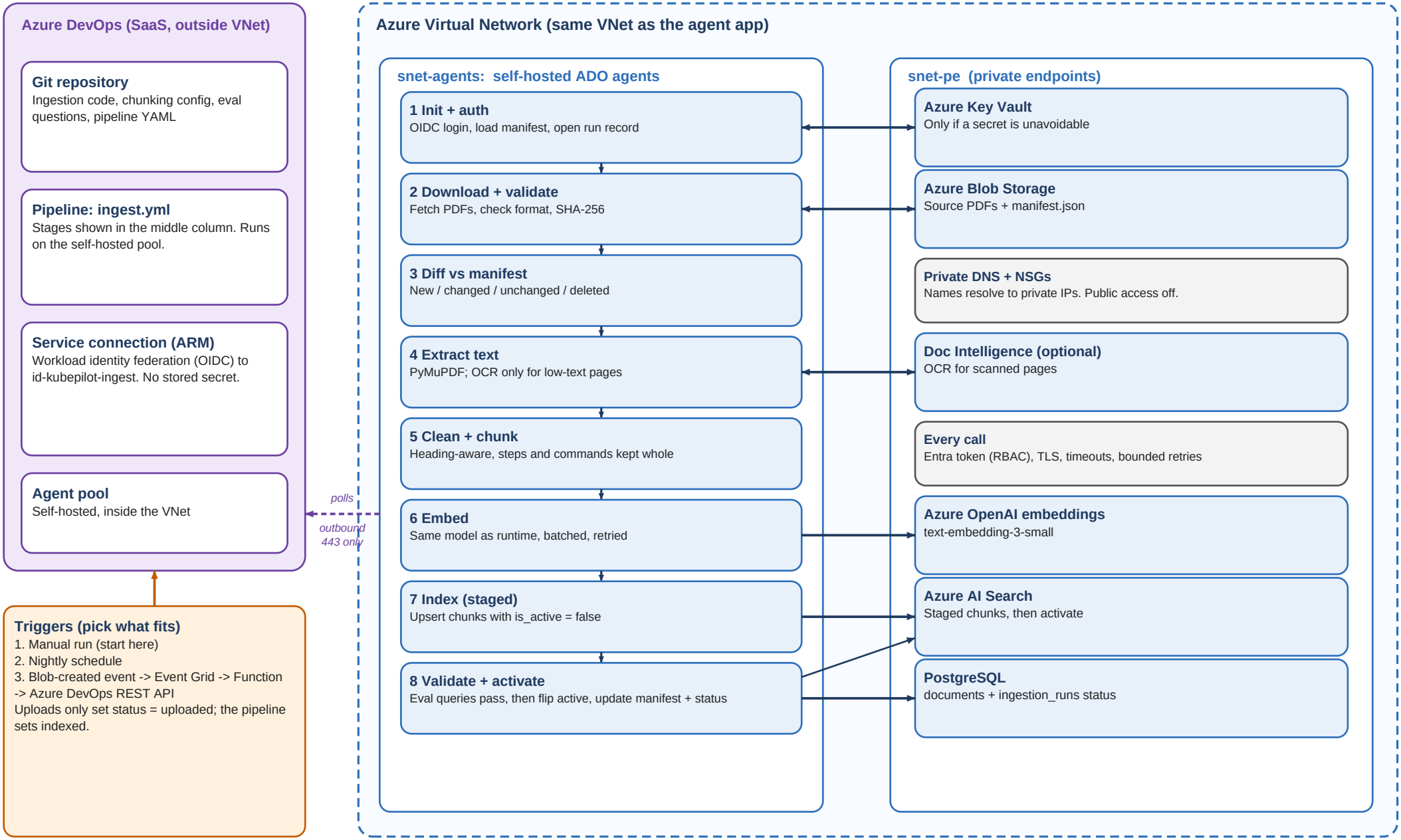
Where the vectors live: Azure AI Search index (one record per chunk)

Field	Type	Purpose
chunk_id (key)	String	Stable id: docid + version + sequence
doc_id, doc_name, version	String / Int	Which document and which version; filterable, shown in citations
status, is_active	String / Bool	draft | approved | retired, and whether this version is the live one. Runtime filters on both
section, page_start, page_end	String / Int	Citation details
content	String (searchable)	Original chunk text, used for keyword search and returned to the LLM
content_vector	Collection(Single), 1536 dims	Embedding for vector search (HNSW, cosine)
allowed_groups	Collection(String), filterable	Entra groups allowed to see this chunk (security trimming)
content_hash, embedding_model, ingested_at	String / Date	Traceability and re-index decisions

Where everything else is stored: PostgreSQL

Table	Main columns	Used by
chat_sessions	id, user_oid, created_at	Backend (session isolation: every query filters by user)
chat_messages	id, session_id, role, content, created_at, trace_id	Backend (short-term memory, bounded by count and retention)
documents	doc_id, name, blob_path, version, sha256, status, ingest_status, uploaded_at, indexed_at	Backend (upload, status in UI), pipeline (sets indexed or failed)
ingestion_runs	run_id, started, finished, embedding_model, counts, result	Pipeline (audit and reporting)
generated_mops	id, title, version, status, blob_path, source_chunk_ids, created_by, created_at	Backend (MOP drafts, always Draft - Pending Human Review)

Ingestion Pipeline Architecture - Azure DevOps, identity and data flow



Ingestion pipeline - stages and failure behavior

#	Stage	What it does	Tools	If it fails
1	Init + auth	Starts the run, logs in through the service connection (OIDC), loads manifest.json from Blob, writes an ingestion_runs row.	Azure CLI task, id-kubepilot-ingest	Run stops. Nothing has changed.
2	Download + validate	Downloads new or changed PDFs. Checks magic bytes, opens as PDF, page count, size, readable text. Computes SHA-256.	Python	Document marked failed validation. Previous active version stays live.
3	Diff vs manifest	Compares path and hash with the manifest: new, changed, unchanged, renamed, deleted. Unchanged files are skipped.	Python, manifest.json	Deleted or renamed sources are retired softly, not purged.
4	Extract text	PyMuPDF with page numbers. Pages with too little text go to OCR. Tables and code blocks are flagged for spot checks.	PyMuPDF, Tesseract or Doc Intelligence	Document marked failed extraction.
5	Clean + chunk	Heading-aware chunks of about 400-800 tokens with 50-100 overlap. Numbered steps and command blocks are never split. Adds metadata.	Python	Chunk count or size outliers raise a warning.
6	Embed	Embeds chunks in batches with the same model as runtime. Records model name and dimensions.	Azure OpenAI embeddings	Retries with backoff. Persistent failure fails that document only.
7	Index (staged)	Upserts chunks for the new version with is_active = false, using stable ids.	Azure AI Search SDK	Old version is untouched and still searchable.
8	Validate + activate	Checks counts, runs eval queries that must return the expected documents, then sets the new version active and the old one inactive. Updates manifest, documents status and the run report.	Search, Blob, PostgreSQL	Not activated. Status set to failed with a reason. Old version stays live.

A separate cleanup job (weekly) deletes inactive old versions after a retention period, so rollback to the previous version stays possible for a while.

Pipeline structure and triggers

Topic	Design
Two pipelines	Ingestion pipeline (this page). Separate app CI/CD pipeline that builds the two images, pushes to ACR and deploys to AKS from the same self-hosted agents (needed because the API server is private).
Why self-hosted agents	Microsoft-hosted agents run outside your VNet and cannot reach private endpoints or a private AKS API server. Agents in snet-agents can. They only connect outbound to Azure DevOps on 443.
Triggers	Start with manual runs while learning. Add a nightly schedule. Later, a Blob-created event can start the pipeline through Event Grid and a small Function, so users get fast indexed status.
Activation rule	A document becomes searchable only after stage 8 succeeds. Upload alone never makes it searchable.

Ingestion - identity, network and what to verify before building

Pipeline identity and connections

Connection	How it authenticates	Permission (least privilege)
Azure DevOps to Azure	Azure Resource Manager service connection using workload identity federation (OIDC) to id-kubepilot-ingest (app registration or user-assigned managed identity). No client secret.	Granted only the roles below
Agent to Azure DevOps	Self-hosted agent polls outbound over 443	Agent pool permissions on the project
Pipeline to Blob	Entra token via service connection	Storage Blob Data Contributor on source and manifest containers (read PDFs, write manifest)
Pipeline to Azure OpenAI	Entra token	Cognitive Services OpenAI User
Pipeline to AI Search	Entra token	Search Service Contributor (create or update index) and Search Index Data Contributor (write chunks)
Pipeline to Doc Intelligence	Entra token (only if used for OCR)	Cognitive Services User
Pipeline to PostgreSQL	Entra database role or password in Key Vault	Write only to documents and ingestion_runs

Subnets

Subnet	Contains	Notes
snet-aks	AKS nodes, internal ingress (private IP), frontend and backend pods	Private cluster: API server has a private IP. Deploy and kubectl only from VNet agents or VPN. Users reach the ingress by VPN, ExpressRoute or a WAF in front (optional snet-appgw).
snet-pe	Private endpoints for Azure OpenAI, AI Search, Blob, Key Vault, ACR and PostgreSQL (if using private endpoint mode)	Private DNS zones linked to the VNet. NSG rules. Public network access disabled on each service.
snet-agents	Self-hosted Azure DevOps agents (VM scale set or Managed DevOps Pool)	Outbound 443 to Azure DevOps and Entra; can reach snet-pe and the AKS API server.

Verify before building (do not assume)

Check	Why it matters
Which Azure OpenAI region, chat model and embedding model are approved and available	Sets the deployment names, quota and cost. Embedding choice cannot change later without re-indexing.
Azure AI Search tier supports private endpoints (not the free tier) and, if wanted, semantic ranking	Basic tier or above is needed for private access.
PostgreSQL Flexible Server network mode: private endpoint or VNet-integrated delegated subnet	Chosen when the server is created. A delegated subnet needs its own subnet.
Whether a private AKS cluster and VNet already exist, and their subnet sizes and DNS setup	Private DNS zones must resolve from AKS and from the agents.
Self-hosted agent pool availability in Azure DevOps, and who may create service connections	Required for private resources.
Entra app registrations, groups and the group-to-document mapping for allowed_groups	Security trimming cannot be added later without re-indexing.
Who approves a document (draft, approved, retired) and where that status is recorded	Only approved documents should be indexed as active.